Project "COMMAND CENTER"

Voice-Controlled Personal AI Assistant for Windows 11

Assistant name: Nova

Wake word: "Hey Nova"

Document version: v1.0 (Design Specification)

Date: April 14, 2026

Status: Approved — pending implementation

Document scope

This document is the approved enterprise-grade design specification for the Command Center project. It covers the full system design including:

- Feature catalog across 8 domains (100+ commands)
- System architecture and process model
- Tech stack with rationale
- Resource budget and performance targets
- Security architecture with 7 layered failsafes
- Privacy and data handling guarantees
- Phased development roadmap
- Risks, testing, and success metrics

Reading guidance

Sections are numbered for easy reference. Tables summarize decisions; prose explains the *why*. The document is designed to be read linearly on first pass, then referenced section-by-section during implementation.

Revision control

Authoritative source (markdown): `C:\\Users\\abhij\\.claude\\plans\\nifty-cuddling-wilkes.md`

This PDF is regenerated from the markdown source whenever the design evolves.

Project "COMMAND CENTER" — Voice-Controlled Personal AI Assistant for Windows

Assistant name: Nova · **Wake word:** "Hey Nova"

> *A personal AI assistant for a Windows 11 laptop. Hybrid (local + cloud) AI, wake-word activated, deep system control, enterprise-grade security with layered failsafes. Designed to be powerful without degrading daily laptop performance.*

---

1. Context

Problem: Daily computer use involves hundreds of small, repetitive actions (opening apps, searching files, checking notes, locking the laptop, controlling media, looking up information, managing reminders). Each context switch costs attention. A voice-controlled "command center" collapses these actions into natural speech and automates multi-step workflows.

Why this matters now: Local speech-to-text (Whisper), local LLMs (Llama 3, Phi-3), and mature Windows automation APIs (pywin32, Win32 WMI, PowerShell) make it feasible to build something that would have required a cloud backend five years ago — while keeping privacy and cost under control.

Intended outcome: A background service that:

- Responds to voice (wake-word) or hotkey in under 500ms
- Executes 80%+ of daily commands locally (no internet dependency)
- Escalates complex reasoning to Claude/GPT APIs when needed
- Deeply controls the OS (lock, launch, automate, monitor) within strict safety rails
- Idles at < 3% CPU and < 200MB RAM so it doesn't interfere with normal laptop use
- Never — under any circumstance — can lock the user out of their own machine

Target user: Solo developer/student on a Windows 11 laptop. English + Hindi capable. Primary device. Must coexist with heavy IDE/browser usage.

Out of scope (for v1): Android companion app, team/multi-user features, cloud-hosted backend, enterprise SSO integration. (Android is a planned v2.)

---

2. Locked Design Decisions

From requirements gathering:

Processing model	Hybrid	Wake-word + intent classification local; complex AI via cloud API
Primary language	Python 3.11+	Mature voice/AI ecosystem, pywin32, fast prototyping
Activation	Wake-word + Hotkey	Porcupine/openWakeWord for "Hey Nova" + `Ctrl+Shift+Space`
System control depth	Deep (admin-capable)	Lock, shutdown, WiFi, brightness, process control, etc.
Security model	Voice auth + PIN + Windows Hello fingerprint + physical escape hatch	Layered, always recoverable, biometric convenience
Master on/off toggle	Tray toggle + `Ctrl+Alt+F11` hotkey, requires active Windows session	Single kill switch for entire system — no audio, no processing, no hotkeys when OFF

---

3. Goals & Non-Goals

Goals

- **G1:** Sub-500ms response latency for local commands (wake word → acknowledgment).
- **G2:** < 3% idle CPU, < 200MB idle RAM, < 1% battery drain per hour idle.
- **G3:** 100+ built-in commands across 10 domains.
- **G4:** Extensible — user can register custom commands without restarting service.
- **G5:** Works fully offline for ~70% of commands (local STT + local intent + OS control).
- **G6:** Zero permanent lockout risk. Physical keyboard escape hatch always honored.
- **G7:** All sensitive actions require voice match *and* (PIN OR Windows Hello fingerprint).
- **G8:** A single, obvious master toggle turns the entire system on/off — accessible only while the Windows user is logged in.

Non-goals

- **N1:** Not a replacement for Cortana/Copilot — complements them.
- **N2:** Not a productivity suite — integrates with existing tools (Notion, Todoist, etc.).
- **N3:** Not multi-user. Single speaker, single machine, single account.
- **N4:** Not a continuous surveillance tool — no meeting recording without explicit "start recording" command.

---

4. Feature Catalog

Organized by domain. **[P0]** = must-have for v1, **[P1]** = first expansion, **[P2]** = advanced/nice-to-have.

4.1 Voice & NLP Engine

- **[P0]** Wake-word detection ("Hey Nova", "Computer") — offline, Porcupine/openWakeWord
- **[P0]** Speech-to-text via local Whisper (base.en, ~150MB, runs on CPU in ~1-2s)
- **[P0]** Text-to-speech via Edge-TTS (free, neural-quality, online) + pyttsx3 offline fallback
- **[P0]** Hotkey activation (`Ctrl+Shift+Space`) for push-to-talk when wake-word isn't ideal
- **[P0]** Intent classifier — rule-based + small embedding model (sentence-transformers MiniLM, ~80MB)
- **[P1]** Multi-turn conversational memory (last 5 turns in-session)
- **[P1]** Speaker verification (anti-spoof) — resemblyzer, ~3ms per utterance
- **[P1]** Bilingual support — auto-detect English/Hindi per utterance (Whisper handles natively)
- **[P1]** Barge-in — interrupt TTS when user speaks
- **[P2]** Custom wake-word training (10 samples → personal wake word)
- **[P2]** Emotion/urgency detection from voice tone (triggers different response style)
- **[P2]** Whisper prompt biasing for your technical vocabulary (repo names, stack, jargon)

4.2 System Control

- **[P0] Lock device** — `rundll32.exe user32.dll,LockWorkStation` (instant, no admin needed)
- **[P0]** Launch any app — indexed registry + Start Menu shortcuts + PATH
- **[P0]** Kill process by name ("kill Chrome", "force close Slack")
- **[P0]** Volume, brightness, mute, media play/pause/skip
- **[P0]** WiFi toggle, Bluetooth toggle, Airplane mode, Night Light toggle
- **[P0]** Sleep / hibernate / restart / shutdown (with 10-second spoken countdown for confirmation)
- **[P1]** Window management — snap left/right, minimize all, virtual desktop switching
- **[P1]** Multi-monitor control — mirror, extend, primary switch (via `DisplaySwitch.exe`)
- **[P1]** Power plan switching ("battery saver", "performance mode")
- **[P1]** Open file/folder by fuzzy name (via Everything SDK — near-instant)
- **[P1]** Run PowerShell/CMD command by voice (sandbox + confirmation required)
- **[P2]** App-specific macros (VS Code palette commands, Chrome tab ops, Spotify control)
- **[P2]** Git automation ("commit staged with message X", "push current branch", "which branch am I on")
- **[P2]** Clipboard history (50 items, voice-recallable: "paste the second-last thing")

4.3 Productivity & Information

- **[P0]** Quick notes (append to daily markdown file)
- **[P0]** Reminders with NLP parsing ("remind me in 30 minutes", "tomorrow at 9am")
- **[P0]** Time, date, timezone queries

- **[P0]** Web search — opens browser to first result or reads snippet
- **[P1]** Calendar integration (Google Calendar via OAuth — read-only first, write in P2)
- **[P1]** Email triage — "any urgent emails?" (Gmail API, read-only)
- **[P1]** Weather briefing, news briefing, stock check (free APIs)
- **[P1]** Unit conversion, calculator, currency conversion
- **[P1]** Pomodoro timer with voice control
- **[P2]** Meeting transcription + summary (on-demand, explicit command)
- **[P2]** Daily stand-up briefing (calendar + emails + news + weather in one "good morning")
- **[P2]** Auto-journal — end-of-day activity summary from logs
- **[P2]** Knowledge base search (local Obsidian/Markdown folder + semantic search)

4.4 AI Intelligence (Cloud-escalated)

- **[P0]** "Ask Nova [anything]" → routes to Claude API with conversation context
- **[P1]** Code helper — "explain this function" (reads clipboard/selection), "refactor", "find bug"
- **[P1]** Document QA — point at PDF, ask questions (Claude with PDF support)
- **[P1]** Screenshot analysis — "what's on my screen" → vision model
- **[P2]** Context injection — assistant sees recent commands + open apps when answering
- **[P2]** Self-improving suggestions — "you opened these 4 apps 50 times this month — want a workflow?"

4.5 Security & Privacy Commands

- **[P0] Master ON/OFF toggle** — single switch in tray icon + `Ctrl+Alt+F11` hotkey. When OFF: no audio capture, no wake-word, no hotkeys (except the toggle-on itself), mic released. State persists across restarts. Toggling requires Windows Hello fingerprint OR PIN (so a bystander can't silently disable it).
- **[P0] Windows Hello fingerprint auth** — integrated via WinRT `UserConsentVerifier`. Works as a drop-in substitute for PIN on all Tier 2 commands and as a recovery factor on lockout.
- **[P0]** "Lock everything" — locks workstation + pauses assistant
- **[P0]** Voice auth + (PIN OR fingerprint) required for sensitive commands (shutdown, file delete, shell exec, credential fetch)
- **[P0]** Panic word ("operation abort") — halts any in-flight action immediately
- **[P1]** Intruder mode — on 3 failed voice auths, capture webcam frame + write to encrypted log
- **[P1]** Bitwarden CLI integration — "get my GitHub password" (voice + fingerprint/PIN required, clipboard auto-clears in 30s)
- **[P1]** File encryption on demand (age or VeraCrypt wrapper)
- **[P2]** Auto-lock on presence loss (webcam face detection — optional, privacy-toggleable)

4.6 Smart Context & Workflows

- **[P0]** Clipboard-aware ("search this on Google", "translate this")
- **[P0]** Workflow macros — "start work mode" → launches IDE + browser + Spotify + DND on
- **[P1]** Screen-aware — OCR the active window, answer questions about it ("what does this error

mean")

- **[P1]** Time-of-day context — "good morning", "good night", "back to work" trigger different flows
- **[P1]** Focus mode — block distracting sites via hosts file, enable DND, Pomodoro-sync
- **[P2]** Activity-aware automation — "he always opens Slack after VS Code on weekday mornings" → suggest workflow

4.7 Third-Party Integrations (opt-in each)

- **[P1]** Spotify / YouTube Music — play, pause, skip, playlist switch
- **[P1]** GitHub — create issue, check notifications, PR status (gh CLI wrapper)
- **[P1]** Notion / Obsidian — quick-capture to daily note
- **[P1]** Todoist / TickTick — add task by voice
- **[P2]** Slack — send message to pinned contact, check DMs
- **[P2]** Google Drive / OneDrive — fuzzy file open

4.8 System Watchdog

- **[P1]** "How's my laptop?" → CPU/RAM/disk/battery/temp summary
- **[P1]** Proactive alerts — low battery (<15%), low disk (<5GB), high temp (>85°C), memory pressure
- **[P1]** Battery health report (from `powercfg /batteryreport`)
- **[P2]** Anomaly detection — unexpected new process, unusual network activity, unknown USB

---

5. System Architecture

5.1 Component diagram (textual)

```
┌──────────────────────────── FOREGROUND (tray icon + GUI settings) ────────────────────────────┐
│                                                                                                 │
│   ┌──────────────────────────── COMMAND CENTER CORE (background service) ──────────────────┐   │
│   │                                                                                         │   │
│   │   ┌──────────────┐    ┌─────────────────┐    ┌──────────────┐    ┌────────────────┐   │   │
│   │   │ Wake Word    │──▶│ Voice Activity  │──▶│ STT          │──▶│ Intent         │   │   │
│   │   │ (Porcupine)  │    │ Detection       │    │ (Whisper)    │    │ Classifier     │   │   │
│   │   └──────────────┘    └─────────────────┘    └──────────────┘    └────────┬───────┘   │   │
│   │          ▲                                                               │           │   │
│   │          │                                                               ▼           │   │
│   │   ┌──────┴───────┐                                          ┌──────────────┐         │   │
│   │   │ Hotkey Agent │                                          │  Router      │         │   │
│   │   └──────────────┘                                          └──────┬───────┘         │   │
│   │                                                                    │                 │   │
│   │      ┌──────────┬──────────┬──────────┬──────────┬──────────┴──────────┐           │   │
│   │      ▼          ▼          ▼          ▼          ▼                     ▼           │   │
│   │   ┌──────┐   ┌──────┐   ┌──────┐   ┌──────────┐  ┌──────────┐                      │   │
│   │   │ OS   │   │ Prod. │   │ AI   │   │ Security │  │ Plugins  │                      │   │
│   │   │ Ctrl │   │ Module│   │ Cloud│   │ Gate     │  │ (custom) │                      │   │
│   │   └──────┘   └──────┘   └──────┘   └──────────┘  └──────────┘                      │   │
│   │      │          │          │          │          │                                  │   │
│   │      └──────────┴──────────┴──────────┴──────────┴──────────┴──────────┴──────────┴──┘   │
│   │                                   ▼                                                     │
│   │                              ┌──────────────┐                                           │
│   │                              │              │                                           │
```

```
│   │                        │ Response Engine │──▶ TTS ──▶ Speaker                    │   │
│   │                        └─────────────────┘                                        │   │
│   │                                                                                   │   │
│   └───────────────────────────────────────────────────────────────────────────────────┘   │
│                                                                                           │
│   Audit Log (encrypted)     Config (YAML)      Command Registry (hot-reloadable plugins)  │
└───────────────────────────────────────────────────────────────────────────────────────────┘
```

5.2 Process model

- **Single Windows Service** (auto-starts, restarts on crash) — runs under user account, NOT SYSTEM. Admin-requiring subcommands use a separate elevated helper invoked via signed IPC.
- **Thread layout:**
  - T1: Audio capture + wake-word (pyaudio, ~1-2% CPU)
  - T2: STT worker (Whisper — dormant until invoked, spikes to ~60-80% one core for 1-2s)
  - T3: Router / intent / plugin execution (event loop)
  - T4: Watchdog / scheduled tasks (reminders, proactive alerts)
  - T5: Tray UI / IPC (Qt or pystray)

5.3 Data flow (typical command)

```
"Hey Nova, lock my laptop"
   │
   ├─ (t=0ms)    Wake word detected → VAD opens mic
   ├─ (t=50ms)   "lock my laptop" captured
   ├─ (t=1200ms) Whisper transcription complete
   ├─ (t=1230ms) Intent: system.lock, confidence 0.98
   ├─ (t=1240ms) Policy check: lock = not-sensitive, no PIN needed
   ├─ (t=1260ms) Execute: LockWorkStation()
   ├─ (t=1280ms) TTS: "Locked." (queued, played as screen locks)
   └─ (t=1300ms) Log: {cmd: "lock", result: "ok", duration: 1.3s}
```

Local-only commands finish in ~1.5s. Cloud-escalated queries add 600ms-2s for API round-trip.

5.4 Plugin architecture

Commands live as Python files in `~/.command-center/plugins/`. Each exposes:

```
# ~/.command-center/plugins/my_custom.py
from command_center.plugin import Plugin, intent

class SpotifyPauseWhenCoffeeBrewing(Plugin):
    @intent(patterns=["coffee is brewing", "starting coffee"],
            requires_pin=False, requires_admin=False)
    def handle(self, ctx):
        ctx.os.run_shortcut("spotify_pause")
        ctx.speak("Paused. Enjoy the coffee.")
```

Hot-reloaded on file save (no service restart). Plugins run in a restricted `ProcessPool` worker — can't access core state or credentials directly, must go through `ctx` API which enforces policy.

---

6. User Interface

The Nova UI is a Windows 11 Fluent-style desktop app built on PySide6 + QtFluentWidgets (Windows 11 native-looking widgets). It is composed of five distinct surfaces, each with a specific role. All UI code runs in the interactive desktop session of the logged-in user — never from the SYSTEM account — so it is automatically unavailable on the lock screen or when the user is signed out.

6.1 Surfaces at a glance

Tray icon	At-a-glance status + instant master toggle	Always, while service runs
Main Dashboard	Full settings, info, audit, plugins, integrations	On demand (tray → Open Dashboard)
Floating Overlay	Live feedback during voice interaction	Only while listening/processing
First-Run Wizard	One-time enrollment (voice, PIN, fingerprint, recovery)	First launch only
Toast Notifications	Reminders, alerts, budget warnings, intruder events	Event-driven

6.2 Tray Icon

The fastest way to interact with Nova — one icon on the taskbar tray.

Visual states:

- **Green solid** — ON and idle (ready to wake)
- **Grey solid** — OFF (master toggle)
- **Blue pulsing** — currently listening to a voice command
- **Purple pulsing** — processing (STT transcribing or cloud call in flight)
- **Red solid** — error state (mic disconnected, API key invalid, etc.)

Left click: toggles master ON/OFF. OFF is instant (so you can silence mid-call). ON requires fingerprint or PIN (§9.6).

Right click menu:

- Status header — "ON — ready" / "OFF" / "Error: [reason]"
- Last 5 commands — click to open full audit log at that row
- Pause for → 30 min / 1 hour / Until tomorrow / Until I re-enable
- Open Dashboard
- Open Settings (jumps to Dashboard → appropriate tab)
- View Audit Log
- Check for Updates

- Quit Nova (stops service; reopen via Start menu)

6.3 Main Dashboard

A single window with a left sidebar (collapsible icons + labels) and a tabbed content area. Minimum size 900×600, resizable, remembers last position/size. Dark mode default, light mode toggle in header.

Thirteen tabs, in sidebar order:

1. Home

- Large status card — ON/OFF toggle + live microphone level meter
- KPIs for today — commands executed, success %, p50 latency, cloud API spend
- Live resource gauges — CPU %, RAM MB, battery impact estimate (per hour)
- Scrolling feed — last 10 commands (time · utterance · intent · auth factor · result)
- System health snapshot — CPU, RAM, Disk free, Battery %, Temperature
- Quick actions row — "Test microphone", "Re-enroll voice", "Open plugins folder", "Force reload plugins"

2. Commands

Browse every registered voice command.

- Searchable, filterable list (by domain, by tier, by enabled state)
- Per-command row: pattern examples, auth tier (T0/T1/T2/T3), enabled toggle, last used, call count
- Click for detail view — edit patterns, test with typed input, re-train voice match, override tier, view source if plugin
- Favorites — starred commands appear in the tray quick-menu

3. Activity Log

Full searchable audit trail (§9.5).

- Timeline view with date-range filter, tier filter, success/fail filter, auth-factor filter
- Columns: timestamp, utterance text (raw audio never retained), intent, auth used (V / V+P / V+F / K+P / K+F), duration, result, error (if any)
- Export to CSV, Export to JSON
- "Clear entries older than N days" (confirmation required)

4. Voice & Audio

- Wake word selector — "Hey Nova" / "Computer" / Custom (record 10 samples to train)
- Voice enrollment status — "Trained on N samples" with "Re-enroll now" button
- Speaker verification toggle + sensitivity slider
- Language mode — English / Hindi / Auto-detect
- TTS voice selector (Edge-TTS neural voices — Aria, Guy, Neerja, etc.) with per-voice preview button
- TTS speed slider (0.8× – 1.5×)
- Microphone selector + input level meter + test recording playback

- Speaker selector + TTS test playback
- Barge-in toggle (can interrupt Nova mid-sentence)

5. Security

Maps directly to §9 Security Architecture.

- PIN — Change (requires fingerprint OR current PIN)
- Windows Hello — Enrollment status, Re-enroll, Fallback-to-PIN toggle
- Recovery code — View (masked), Regenerate, Print, Export to USB (each requires fingerprint)
- Per-command tier override — promote/demote individual commands
- Failed attempts log — last 30 days, with auth factor tried
- Panic word — configure (default: "operation abort")
- Intruder mode — toggle + settings (N failed attempts → webcam capture)
- Presence detection — webcam face detection → auto-lock on walk-away (opt-in, off by default)

6. Plugins

- Installed plugins table — name, version, enabled toggle, last error, avg runtime, memory
- Plugin source viewer — opens the `.py` file in your default editor
- "Open plugins folder" shortcut → `~/.command-center/plugins/`
- "Reload all plugins" (hot-reload without service restart)
- Install from folder / from verified manifest URL
- Per-plugin sandbox resource limits (CPU %, RAM MB, timeout seconds)

7. AI & Cloud

- Claude API key — status indicator (stored in Windows Credential Manager, never displayed)
- Monthly spend gauge — `$X.XX / $Y.YY` progress bar with color changes at 80% / 100%
- Set monthly budget + alert thresholds
- Model selector — Claude Sonnet 4.6 / Opus 4.6 / Haiku 4.5
- Context injection preferences — include clipboard, include recent commands, include open app list (each togglable)
- Offline fallback model — toggle + install button (downloads Llama 3.2 3B, ~2GB, one-time)
- API call log — timestamp, tokens in/out, cost estimate, success/failure

8. Integrations

Card-based layout. Each integration has: icon, status, Connect/Disconnect button, scope display.

- Google Calendar
- Gmail
- Spotify / YouTube Music
- GitHub (via `gh` CLI)
- Notion / Obsidian
- Todoist / TickTick
- Bitwarden

- Weather provider
- News provider
- Slack (P2)

Each card shows: connection status (green/grey), scope granted (read-only / read-write), OAuth token expiry date, Disconnect button.

9. Automations / Workflows

A simple visual workflow builder — not a full Zapier.

- List of workflow macros — "start-work-mode", "morning-briefing", "end-day", user-created
- Drag-to-reorder steps within a workflow
- Step types — launch app, run voice command, speak text, wait N sec, toggle setting, call integration API
- Triggers — time-of-day, voice command, manual
- Test-run button (dry run with verbose log, no actual execution)

10. Reminders & Notes

- Active reminders list (sortable by time, category)
- Add/edit/delete — with NLP parse preview ("remind me tomorrow at 9am to..." → resolved time displayed before save)
- Notes folder location — default `Documents/Nova/notes/`, configurable
- Recent daily notes preview (read-only; full editing in your editor of choice)

11. Performance & Resources

Critical for the "doesn't slow my laptop" guarantee (§8).

- Live graphs — CPU %, RAM MB over last 60 minutes
- Battery-aware thresholds — power-saver mode activates at N% battery (default 25%)
- Auto-throttle CPU threshold (default 70%)
- Wake-word sensitivity slider (higher = fewer false positives, more misses)
- STT model selector — tiny (39MB) / base (150MB, default) / small (480MB)
- "Unload STT after N seconds idle" (default 60s)
- Max plugin process memory (default 100MB)

12. Advanced

- Launch on Windows boot (ON by default)
- Service control — Start / Stop / Restart / View service status (Windows Services snap-in link)
- Log level — Debug / Info / Warning / Error
- Hotkey rebind — `Ctrl+Shift+Space` (push-to-talk), `Ctrl+Alt+F11` (master toggle), `Ctrl+Alt+F12` (kill switch)
- Export all settings to JSON (for backup / transfer to another laptop)
- Import settings from JSON
- Factory reset — requires recovery code; wipes all Nova data but leaves Windows untouched

- About — version, dependencies, licenses, Check for Updates button

13. Help

- Quick-start guide (embedded markdown renderer)
- Full command reference (searchable + printable)
- Troubleshooting common issues
- Keyboard shortcuts cheatsheet
- Report an issue (opens GitHub issue form in browser; user fills out manually)

6.4 Floating Overlay

A small translucent bar at the bottom-center of the primary display, visible **only** while Nova is actively listening or processing. ~400 × 60 px, click-through-to-cancel.

Content:

- Pulsing microphone icon with live audio level
- Live transcription text (updates as Whisper processes)
- Intent badge once classified (e.g., `system.lock` in blue, `shell.exec` in orange for Tier 2)
- Duration counter (ms)
- Cancel button (also `Esc` key)

Dismissal: auto-hides 2 seconds after response is spoken.

Why it matters: gives you instant visual confirmation of what Nova heard before it acts — especially critical for Tier 2 destructive commands, where you see `system.shutdown` displayed *before* the fingerprint prompt appears.

6.5 First-Run Setup Wizard

A one-time guided flow on first launch. Ten steps, ~5 minutes total. The service will not start voice processing until the wizard completes.

1. **Welcome** — overview of Nova, privacy promises, what data stays local
2. **Permissions** — request microphone access, notification permissions
3. **Audio test** — select mic, record + playback test; select speaker, TTS test
4. **Voice enrollment** — read 5 on-screen sentences aloud (captures speaker embedding)
5. **PIN setup** — choose 6-digit PIN, confirm twice
6. **Windows Hello fingerprint** — enroll via native Windows dialog (skippable if no sensor, can enroll later)
7. **Recovery code** — displayed on screen + mandatory "save to file" OR "print" before Next button enables
8. **Wake word** — pick "Hey Nova" (default) or "Computer"; custom wake word enrollable later
9. **Claude API key** — paste key (optional, can skip); link to Anthropic Console for key creation
10. **You're ready** — suggested first command ("Hey Nova, what time is it"), then launches Dashboard

6.6 Toast Notifications

Standard Windows 11 toast notifications via the `winrt` notification API:

- Reminders firing (with Snooze / Dismiss actions)
- Low battery (<15%), low disk (<5GB), high CPU temp (>85°C) alerts
- Cloud API budget warnings — at 80% and 100% of monthly cap
- Intruder alerts — with webcam thumbnail if intruder mode enabled
- Update available
- Plugin crash / recovery events (only if the plugin doesn't auto-recover)

User can disable each category in Dashboard → Advanced → Notifications.

6.7 Design Language

- **Windows 11 Fluent** — acrylic/mica blur on title bar, rounded corners, reveal hover effects
- **Dark mode default**, light mode toggle in header; follows Windows system preference on first launch
- **Accent color** inherits from Windows system accent, with override option
- **Typography** — Segoe UI Variable for body, Cascadia Code for code
- **Keyboard accessibility** — all controls tab-navigable, every button has an `Alt+` shortcut, Esc always closes modals
- **Responsive** — window scales from 900×600 minimum to fullscreen; sidebar collapses to icons-only under 800 px width
- **Internationalization-ready** — string table externalized, starts English-only, Hindi localization in P2

6.8 UI performance budget

The UI itself follows the same resource discipline as the service:

- Dashboard idle (open but unfocused): ≤ 50 MB RAM, ≤ 0.5% CPU
- Dashboard active (foreground, graphs updating): ≤ 120 MB RAM, ≤ 2% CPU
- Tray icon + hidden dashboard: ≤ 20 MB RAM, ≤ 0.1% CPU
- Live graphs update at 1 Hz (not 60 Hz) to conserve battery
- Dashboard fully closed: process exits, only tray + service remain

---

7. Tech Stack

Language	Python 3.11+	Ecosystem, speed to ship
Wake word	`pvporcupine` (free tier) or `openWakeWord`	Offline, <2% CPU
VAD	`webrtcvad`	Standard, lightweight
STT (local)	`faster-whisper` base.en	4x faster than whisper.cpp on CPU

STT (cloud fallback)	Deepgram Nova-2 or OpenAI Whisper API	When local is too slow or noisy
Intent	`sentence-transformers` MiniLM + rule engine	Fast, accurate for closed-set commands
LLM (cloud)	Claude Sonnet 4.6 via Anthropic SDK	Best reasoning, 200K context
TTS	`edge-tts` (primary) + `pyttsx3` (offline)	Free, neural quality, offline fallback
OS control	`pywin32`, `pyautogui`, `psutil`, `WMI`	Complete Windows surface
Windows Hello	`winrt-Windows.Security.Credentials.UI`	Fingerprint/face auth via `UserConsentVerifier`
Session detection	`pywin32` (`win32ts`)	`WTSQuerySessionInformation` for logged-in-user check
Global hotkey	`keyboard` or `pynput`	Low-level hook
Tray icon	`pystray`	Native Windows tray, lightweight
Dashboard/ Settings UI	`PySide6` + `PySide6-Fluent-Widgets` (QtFluentWidgets)	Native Windows 11 Fluent look — acrylic, rounded corners, reveal hovers
Charts (resource graphs)	`pyqtgraph`	Fast, low-overhead real-time plots
Toast notifications	`winrt-Windows.UI.Notifications`	Native Windows 11 toasts
Service host	`pywin32` service wrapper or NSSM	Auto-start, crash recovery
Config	YAML + Pydantic validation	Human-editable, typed
Storage	SQLite (commands log, reminders) + AES-encrypted file (secrets)	Zero-infra, portable
Packaging	PyInstaller one-file EXE + Inno Setup installer	End-user install
Logging	`loguru` → rotating encrypted log	Easy debug, private

---

8. Resource Budget

Non-negotiable targets. If we can't stay under these, we redesign.

CPU (avg)	≤ 3%	≤ 25% burst (STT), 2s max	40% burst
RAM	≤ 200 MB	≤ 600 MB (STT loaded)	800 MB
Disk (install)	—	—	≤ 1.5 GB (incl. Whisper model)
Disk (data/logs)	—	—	≤ 500 MB (logs auto-rotate)
Network (idle)	0	—	—
Network (cloud query)	—	≤ 50 KB per query	—
Battery impact	≤ 1%/hr idle	—	≤ 2%/hr idle
Startup time	—	—	≤ 3s to wake-word ready

How we stay under these:

- **Lazy loading:** Whisper model loads only on first invocation, unloads after 60s idle (saves 300MB RAM when quiet).
- **Battery-aware:** On battery < 25% or power-saver mode, wake-word detection drops to 8kHz (halves CPU) and hotkey becomes primary.
- **Smart scheduling:** Scheduled tasks (reminders check, log rotation) batch every 60s, not continuously.
- **Single-core affinity:** All always-on threads pinned to core 0; bursts allowed to scatter.
- **Automatic throttle:** If system CPU > 70% for 10s, assistant pauses STT and falls back to hotkey mode only.
- **Memory cap enforcement:** `psutil` self-monitors; if RSS > 800MB, restart subprocess.

---

9. Security Architecture

9.1 Threat model

Someone walks up, says "delete all my files"	Med	Critical	Voice auth + (PIN OR fingerprint) on destructive actions
Recorded voice playback attack	Low	Critical	Anti-spoof via resemblyzer + liveness challenge (fingerprint or PIN)

Bystander silently disables the assistant	Low	Medium	Master toggle OFF→ON requires fingerprint or PIN
Malicious plugin	Med	High	Plugin sandbox, signed plugin manifest
Assistant executes injected command from audio ad/ podcast	Med	Medium	Wake-word + VAD + short utterance window; destructive actions require fingerprint/PIN
API key leak	Low	High	Keys in Windows Credential Manager, never in config files
Fingerprint sensor spoofed	Very Low	High	Windows Hello has built-in liveness + anti-spoof; voice factor still required (2FA)
Fingerprint sensor hardware fails	Med	Low	PIN path remains; both factors independent
Assistant locks user out	**ZERO-TOLERANCE**	Critical	See §9.3 failsafes (6 recovery paths)

9.2 Authorization tiers

Every command has a tier. The router enforces it. **Authentication factors**: voice match (V), PIN (P), Windows Hello fingerprint (F), physical keyboard (K). Wherever PIN is accepted, fingerprint is interchangeable — user picks per-command in settings.

- **Tier 0 (open):** time query, weather, web search, notes read, launch whitelisted app — *no auth*
- **Tier 1 (voice-match):** lock device, volume, brightness, WiFi, reminders, launch any app — *V*
- **Tier 2 (voice + biometric/PIN):** shell exec, file delete, process kill (system processes), credential fetch, shutdown, "ask cloud AI with my context", **toggling master ON/OFF** — *V + (P or F)*
- **Tier 3 (physical keyboard only):** disable assistant from settings, reset security config, erase logs, change recovery code — *K + (P or F)*

Speaker verification fails open to Tier 0 (so a guest can ask "what time is it") but cannot do anything stateful.

Why fingerprint matters for UX: Tier 2 commands are common (10-20×/day for power users). Typing a PIN every time creates friction; fingerprint via the laptop's built-in sensor is one tap and ~300ms. PIN remains the always-available fallback if the fingerprint sensor is dirty/wet/broken.

9.3 Failsafes — you can NEVER be permanently locked out

This is the single most important design principle. Six redundant recovery paths:

1. **Physical kill switch** — `Ctrl + Alt + F12` held for 3 seconds immediately disables the assistant service. Implemented as a low-level keyboard hook that runs *before* any assistant processing. Cannot be overridden by voice commands or plugin.
2. **Master OFF toggle** — tray icon + `Ctrl+Alt+F11`. Softer than the kill switch (service keeps running, just dormant) but instantly stops all audio/hotkey activity. Toggling back ON requires fingerprint or PIN.

3. **Safe mode boot** — Assistant is a user-mode service, not a driver. Booting Windows in Safe Mode never loads it.
4. **Fingerprint bypass** — If you forget the PIN but the sensor works, Windows Hello fingerprint unlocks everything including Tier 3. If the sensor is broken, the PIN still works. Two independent auth factors, either one sufficient for recovery.
5. **PIN recovery file** — On install, a 20-character recovery code is generated and written to a file at a path you choose (recommend: print + USB). Entering it via the tray icon's "I'm locked out" dialog resets all security — works even if both fingerprint and PIN are lost.
6. **5-attempt decay** — Tier 2 PIN/fingerprint attempts are rate-limited to 5 per 10 minutes. After 5 failures, the assistant self-disables Tier 2 for 1 hour *but does not lock the OS* — the OS is untouched. You can keep working, just without sensitive voice commands.
7. **No assistant action ever locks Windows itself irreversibly** — "Lock my laptop" uses `LockWorkStation` which is the same as `Win+L`; you unlock with your normal Windows password, PIN, or fingerprint (same as always). The assistant has no ability to change your Windows password or Windows Hello config.

> ***Guarantee:*** *The assistant can decline to do things for you. The assistant can never prevent you from using your laptop.*

9.4 Secrets handling

- API keys (Claude, Deepgram, Spotify, GitHub) → Windows Credential Manager via `keyring`.
- OAuth tokens → Credential Manager, encrypted at rest.
- Voice auth embedding → local SQLite, encrypted with key derived from PIN (Argon2).
- Audit log → SQLite with AES-256 encryption (key also in Credential Manager).

9.5 Audit log

Every command logged with:

- Timestamp, utterance text, intent, tier, decision (executed/denied), duration, auth factor used (V / V+P / V+F / K+P / K+F)
- No raw audio retained (deleted after STT)
- Tray icon has "Show last 20 actions" button — always transparent
- Log rotation: 500MB cap, 30-day retention

9.6 Master Enable/Disable Toggle

The master toggle is the "light switch" for the entire project. It exists separately from the kill switch because it serves a different purpose: day-to-day on/off control vs. emergency shutdown.

Access paths (all equivalent):

- Tray icon: left-click → toggle, color changes green/grey
- Hotkey: `Ctrl+Alt+F11` (distinct from kill switch `Ctrl+Alt+F12`)
- Voice command (only when ON → OFF): "Nova, pause yourself" (requires voice+fingerprint/PIN)

Logged-in-user requirement: The toggle is exposed only via user-session UI (tray icon runs in the interactive desktop session) and requires the Windows user to be logged in at the active console. Verified via:

- `WTSQuerySessionInformation` with `WTSConnectState` → must be `WTSActive`
- Session ID of tray UI must match active console session
- If user logs out or switches user: assistant auto-pauses (not fully off, but audio stops); resumes on login

State behaviour when OFF:

- Wake-word detector: stopped, microphone handle released (LED off on laptops that have one)
- Global hotkeys: unregistered (except the F11 toggle-on)
- Scheduled tasks (reminders): **still fire** — reminders are important and non-intrusive (desktop notification only, no voice)
- Watchdog alerts: still fire (critical for battery/disk warnings)
- Plugins: all dormant
- Cloud API calls: disabled

State behaviour when ON:

- Normal full operation
- Tray icon tooltip shows "ON — ready"
- Subtle chime on transition (user-configurable, off by default)

Security on the toggle itself:

- Turning OFF: requires only the logged-in user's session (no auth) — this must be instant so you can shut it up during a call
- Turning ON: requires Windows Hello fingerprint OR PIN (prevents a bystander from covertly re-enabling listening after you turned it off)
- All toggle events logged with auth factor

Startup default:

- On fresh install: OFF (opt-in)
- On subsequent boots: restores last state
- First-run setup wizard guides through enrollment: voice sample, PIN choice, fingerprint registration, recovery code printout

---

10. Privacy & Data

- **Audio:** Captured → transcribed → discarded. Never written to disk. Never sent to cloud unless user explicitly opts into Deepgram/Whisper API (off by default).
- **Transcripts:** Stored locally in encrypted audit log, 30-day retention.
- **Cloud AI queries:** Only the text of the current query + opt-in context (last N commands, clipboard). User sees exactly what's being sent via tray notification "sending to Claude: [preview]".

- **Telemetry:** None. Zero outbound network calls except user-initiated (cloud AI, calendar, integrations).
- **Voice embeddings:** Never leave the device.

---

11. Development Roadmap

Phase -1 — Documentation artifact (pre-code)

Before any code: export this plan as a **polished PDF** for user reference. Stored at: `C:\Users\abhij\Desktop\Study\Projects\command center\docs\command-center-design-v1.pdf`

Generation approach:

- Primary: `pandoc` (if installed) — markdown → PDF with LaTeX engine (best typography)
- Fallback: Python `markdown-pdf` or `weasyprint` (markdown → HTML → PDF)
- Secondary fallback: `reportlab` programmatic PDF
- Output styling: clean serif body, monospace code, syntax-highlighted code blocks, table of contents, page numbers, cover page with project name + version + date

Deliverable: PDF file user can read offline, print, annotate.

Phase 0 — Foundation (setup + scaffolding)

- Python project scaffold, Poetry/uv, CI
- Config + logging + encrypted storage primitives
- Tray icon + settings UI shell with master ON/OFF toggle wired up first
- Windows service wrapper with session-change hooks (pause on logout)
- First-run setup wizard: voice enrollment → PIN → fingerprint registration (via Windows Hello) → recovery code printout
- `Ctrl+Alt+F11` master toggle hotkey + `Ctrl+Alt+F12` kill switch hotkey
- **Deliverable:** service starts, shows in tray, toggle works, all auth factors enrolled, does nothing else

Phase 1 — Voice pipeline (MVP)

- Wake-word detection
- STT (local Whisper)
- TTS (Edge-TTS + pyttsx3 fallback)
- Hotkey activation
- Simple echo command ("Nova, say hello")
- **Deliverable:** round-trip voice interaction works

Phase 2 — Core commands (P0 features)

- Intent router + plugin system
- OS control: lock, launch apps, volume, brightness, WiFi, BT, sleep/shutdown

- Quick notes, reminders, time/date/weather/web-search
- Security gate (Tier 0/1/2), voice auth, fingerprint + PIN dual-factor flows
- Physical kill switch (F12)
- **Deliverable:** daily-driver ready for 20-30 commands

Phase 3 — Intelligence layer (P1)

- Claude API integration, "ask Nova anything"
- Clipboard-aware, screen-aware (OCR)
- Workflow macros
- Calendar + email read-only
- **Deliverable:** feels like a real assistant, not a macro runner

Phase 4 — Polish & advanced (P2)

- Multi-turn memory
- Custom wake-word training
- Integrations (Spotify, GitHub, Notion, Todoist)
- Watchdog + proactive alerts
- Self-improving suggestions
- **Deliverable:** daily indispensable tool

Phase 5 — Android companion (v2, not v1)

- Out of scope for current plan. Push-notification-driven "remote control" app that talks to laptop service over LAN + secure cloud relay.

---

12. Risks & Mitigations

Whisper CPU burst slows IDE	High	Lazy load, auto-unload, pause under system load
Wake-word false positives during calls/ videos	Med	VAD + confidence threshold + user-trainable negative examples
Cloud API costs spiral	Med	Monthly budget cap (default $10), per-command cost estimate shown
Plugin crashes take down service	Med	Plugin sandbox in subprocess, supervisor restarts
Windows update breaks pywin32	Low	Pin versions, CI tests on each Windows update channel

Anthropic API outage	Low	Fallback to local Llama 3.2 3B for simple Q&A (optional install)
User forgets PIN	Med	Fingerprint still works → no action needed; or recovery code → reset
Fingerprint sensor driver breaks after Win update	Med	PIN path still works; PIN never depends on hardware
User loses recovery code AND forgets PIN AND sensor broken	Very Low	`Ctrl+Alt+F12` kill switch → uninstall → reinstall (data preserved in user folder)
Microphone privacy concerns	Med	Hard-off mute button in tray, LED indicator in tray icon when listening

---

13. Verification & Testing

Unit tests

- Intent classifier accuracy on 500-command test set (target: ≥ 95%)
- All OS control functions mocked + integration-tested
- Security policy enforcement (every tier transition has a test)

Integration tests

- End-to-end: synthetic voice → STT → intent → action → TTS (on CI with recorded clips)
- Failsafe: simulate 5 wrong PINs, verify rate-limit kicks in, verify kill switch still works
- Resource: 24-hour idle test, assert CPU/RAM stay in budget

Manual acceptance tests (before each release)

- [ ] Wake word works in quiet room, noisy room, with music playing
- [ ] Hotkey works even when IDE has focus and is CPU-busy
- [ ] "Lock laptop" locks screen; normal Windows password/fingerprint unlocks
- [ ] 5 wrong PINs → Tier 2 disabled 1hr, OS unaffected
- [ ] 5 wrong fingerprint reads → Tier 2 disabled 1hr, PIN path still works
- [ ] Forget PIN + fingerprint works → recovery via fingerprint alone
- [ ] Sensor damaged + PIN works → recovery via PIN alone
- [ ] Forget both → recovery code from USB resets security
- [ ] `Ctrl+Alt+F12` held 3s → service stops, icon grays out
- [ ] `Ctrl+Alt+F11` → master toggle flips; OFF stops mic LED within 500ms
- [ ] Master toggle OFF: wake-word dead, hotkeys dead (except F11), reminders still fire
- [ ] Master toggle ON requires fingerprint/PIN (bystander can't re-enable)

- [ ] Log out of Windows → assistant auto-pauses; log back in → resumes to last state
- [ ] Lock screen (Win+L) → tray UI inaccessible, hotkeys inaccessible
- [ ] Unplug network → Tier 0/1 commands still work
- [ ] 8-hour typical use (IDE + browser + meetings): no noticeable slowdown
- [ ] Cloud AI query → preview shown → user can cancel before send
- [ ] Uninstall removes all files except audit log archive in user-chosen location

Observability

- Local dashboard (tray → "Stats") shows: commands/day, avg latency, error rate, cloud API spend this month, top 10 commands.

---

14. Critical Files / Project Layout

```
command-center/
├── command_center/
│   ├── __init__.py
│   ├── service.py            # Windows service entrypoint
│   ├── voice/
│   │   ├── wake_word.py      # Porcupine wrapper
│   │   ├── stt.py            # Whisper
│   │   ├── tts.py            # Edge-TTS + pyttsx3
│   │   └── vad.py
│   ├── intent/
│   │   ├── classifier.py     # Embeddings + rules
│   │   └── router.py
│   ├── os_control/
│   │   ├── windows.py        # pywin32 wrappers
│   │   ├── apps.py           # Launcher, process ctrl
│   │   ├── media.py          # Volume, brightness
│   │   └── network.py        # WiFi, BT
│   ├── ai/
│   │   ├── claude_client.py
│   │   └── context.py
│   ├── security/
│   │   ├── auth.py           # Voice match + PIN
│   │   ├── hello.py          # Windows Hello (WinRT UserConsentVerifier)
│   │   ├── policy.py         # Tier enforcement (V / V+P / V+F)
│   │   ├── master_toggle.py  # Global ON/OFF + session-change listener
│   │   ├── killswitch.py     # Ctrl+Alt+F12 hook
│   │   └── audit.py
│   ├── productivity/
│   │   ├── notes.py
│   │   ├── reminders.py
│   │   └── calendar.py
│   ├── plugins/
│   │   └── loader.py         # Hot-reload, sandbox
│   ├── ui/
│   │   ├── tray.py              # System tray icon + right-click menu
│   │   ├── dashboard.py         # Main window shell (sidebar + tabs)
│   │   ├── overlay.py           # Floating voice-interaction overlay
│   │   ├── wizard.py            # First-run 10-step setup wizard
│   │   ├── notifications.py     # Windows toast wrapper
│   │   ├── theme.py             # Dark/light + accent color inheritance
│   │   └── tabs/
│   │       ├── home.py          # KPIs, status, live feed
```

```
│   │       ├── commands.py      # Browse + test commands
│   │       ├── activity.py      # Audit log viewer
│   │       ├── voice.py         # Wake word, enrollment, TTS voice
│   │       ├── security.py      # PIN, Hello, recovery, tiers
│   │       ├── plugins.py       # Plugin management
│   │       ├── ai.py            # Claude key, budget, model
│   │       ├── integrations.py  # OAuth cards for each service
│   │       ├── automations.py   # Workflow builder
│   │       ├── reminders.py     # Reminders + notes
│   │       ├── performance.py   # Resource graphs + throttle settings
│   │       ├── advanced.py      # Startup, service, export/import
│   │       └── help.py          # Docs, shortcuts, troubleshooting
│   └── storage/
│       ├── db.py             # SQLite
│       └── secrets.py        # Credential Manager
├── plugins_user/             # User's custom plugins (hot-reloaded)
├── tests/
├── pyproject.toml
├── installer.iss             # Inno Setup
└── README.md
```

---

15. Appendix — Example Command Catalog (v1 target: 100+)

Abbreviated sample — each is one line of config in the intent rule engine.

OS: lock / unlock-hint / sleep / restart / shutdown / log off / volume up|down|mute / brightness up|down / wifi on|off / bluetooth on|off / airplane on|off / night light on|off / focus assist on|off

Apps: open {app} / close {app} / kill {app} / switch to {app} / list running apps / open {app} in {folder}

Files: find file {name} / open folder {name} / open last download / recent files / empty recycle bin (Tier 2)

Windows: snap left / snap right / maximize / minimize all / virtual desktop next|prev / new desktop

Notes: quick note {text} / read today's notes / show yesterday's notes

Reminders: remind me {when} to {what} / list reminders / cancel last reminder

Info: what time / what date / weather / weather tomorrow / stock {ticker} / news briefing

Productivity: start pomodoro / pause pomodoro / how long have I been working / calendar today / next meeting

Web: search {query} / open {site} / translate {text} / define {word}

Media: play / pause / next / previous / volume {N} / what's playing

AI: ask {question} / summarize clipboard / explain this screen / draft email to {person}

Security: lock everything / enable intruder mode / show audit log / change pin (Tier 3 physical) / pause yourself (master OFF, voice) / enroll fingerprint / re-enroll voice

Workflows: start work mode / start focus mode / end day / morning briefing

Dev: open terminal here / git status / git push / create branch {name}

---

16. Success Metrics (measured after 30 days of use)

- **Daily active commands:** ≥ 40 (proves habit formation)
- **Latency p50:** < 1.5s local, < 2.5s cloud
- **Crash rate:** < 1 per week
- **False wake-word activations:** < 5/day
- **Battery impact:** ≤ 1%/hr measurable delta vs baseline
- **User can't-live-without score:** subjective but honest ("would I pay $20/mo for this if someone else made it?")

Next step

On approval of this plan, the very first action (Phase -1) will be to export this document as a polished PDF to: `C:\Users\abhij\Desktop\Study\Projects\command center\docs\command-center-design-v1.pdf`

You will review that PDF. Only after your explicit "proceed" does any code implementation begin at Phase 0 (project scaffold + master toggle + auth enrollment).

Each subsequent phase will have its own smaller plan written before coding starts.